This demo uses a chain of classifiers instantiated with logistic regression as the base classifier via ClassifierChain & LogisticRegression from scikit-learn. This allows us to predict the prob. of each emotion class for each sample with chain_model.predict_proba in Step ⑤.

Subsequently, the train_and_save function fits the classifier on the train set & evaluates it with chain.score. This calculates the mean accuracy relative to the test set. In the last line, the function saves the trained classifier for determining emotion in a pickle file emotion_chain.pkl. A pickle file is a way of serializing & saving Python objects such as trained ML models. By saving a trained model as a pickle file, we can reuse the model later w/o retraining it from scratch.

Then, we save the trained classifier for determining emotion into a pickle file named emotion_chain.pkl. To load the saved model later, we can simply use the pickle.load function in Python.

```
def train_and_save():
    full_df = pd.read_json("./data/xed_with_embeddings.jsonl", orient="index")
    df = full_df
    mlb = MultiLabelBinarizer()
    X = np.array(df.embeddings.tolist())
    y = mlb.fit_transform(df.labels_text)
    classes = mlb.classes_
    classes_mapping = {index: emotion for index, emotion in enumerate(mlb.classes_)}
    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                        test_size = 0.01, random_state = 42)
```

```
base_lr = LogisticRegression(solver='lbfgs',
                             random_state=0)
chain = ClassifierChain(base_lr, order='random',
                        random_state=0)
chain.fit(X_train, y_train)
print(chain.score(X_test, y_test))
pickle.dump(chain, open("./data/models/emotion_chain.
                        pkl", 'wb'))
```

↳ Step③: Get User Input

Streamlit functions facilitate user input. Calling st.text_input creates a page object that asks the user "How are you feeling?" and captures the user's text responses. It then presents the user a slider via st.slider, which they use to select the no. of top emotions they'd like to be presented (k).

```
feeling_text = st.text_input("How are you feeling?", "")
top_k = st.slider("Top Emotions", min_value=1,
                  max_value=len(classes_mapping),
                  value=1, step=1)
```

↳ Step④ Embed the Input

The beginning of function score_sentence embeds the user input text into embeddings & converts it into a torch-based tensor.

```
embeddings = torch.as_tensor(get_embeddings(co=co,
                                model_name = model_name,
                                texts = [text]), dtype= torch.float32)
```

The function imports get_embeddings from the utils.py file. Then, the get_embeddings function calculates the embeddings by calling the embed-multilingual-V3.0 model.

```
def get_embeddings(co: cohera.Client,
                   texts: List[str],
                   model_name: str= 'embed-multilingual-v3.0',
                   input_type ="search_document",
                   truncate: str= 'RIGHT',
                   batch_size: int= 2048) -> List[float]:

                   "blobheart", delay =True

def get_embeddings_api(texts_batch : List[str]):

    for i in range(N_MAX_RETRIES):
        try:
            output= co.embed(model = model_name,
                             input_type= input_type,
                             texts = texts_batch, truncate=
                                                truncate)

            break
        except Exception as e:
            if i == (N_MAX_RETRIES-1):
                print(f"Exceeded max retries with
                                error {e}")
                raise f"Error {e}"

    return output.embeddings
```

```
st_pbar = tqdm(range(0, len(texts), batch_size))
for index in st_pbar:
    texts_batch = texts[index : index + batch_size]
    embeddings_batch = get_embeddings_api
                        (texts_batch)
                # List(pool.imap(get_embeddings_api,
                        [texts_batch]))

    embeddings.append(embeddings_batch)

return np.concatenate(embeddings, axis=0).tolist()
```

↳ Step 5: Classify Sentiment

- In the function setup within the sentiment.py file, you can access the trained classifier model from emotion_chain.pkl where we saved it in Step ①.

```
def setup():
    ...
    chain_model = pickle.load(open(model_path, 'rb'))
```

With the model stored as chain_model, we access it in score_sentence after acquiring the embeddings. The function then executes the model on the float tensor of embeddings to predict probs of emotion class(es), determined for each sample with chain_model.predict_proba.

Then, torch.sort sorts the probability O/Ps for the emotion(s) associated with the user text input in ascending order.

In the last
& tensor of
CPU-based, allo
into a Numpy
arrays such t
is first. Thi
the images a
emotions exp

outputs =

probas
indices

↳ Section 6:

Let's 1st head
Step ⑤ : The 1
mapping as
are mapped
Corresponding
emotion2i

emotion g

```
st_pbar = tqdm(range(0, len(texts), batch_size))
for index in st_pbar:
    texts_batch = texts[index : index + batch_size]
    embeddings_batch = get_embeddings_api
                        (texts_batch)
                # List(pool.imap(get_embeddings_api,
                        [texts_batch]))

    embeddings.append(embeddings_batch)

return np.concatenate(embeddings, axis=0).tolist()
```

↳ Step 5: Classify Sentiment

- In the function setup within the sentiment.py file, you can access the trained classifier model from emotion_chain.pkl where we saved it in Step ①.

```
def setup():
    ...
    chain_model = pickle.load(open(model_path, 'rb'))
```

With the model stored as chain_model, we access it in score_sentence after acquiring the embeddings. The function then executes the model on the float tensor of embeddings to predict probs of emotion class(es), determined for each sample with chain_model.predict_proba.

Then, torch.sort sorts the probability O/Ps for the emotion(s) associated with the user text input in ascending order.

In the last
& tensor of
CPU-based, allo
into a Numpy
arrays such t
is first. Thi
the images a
emotions exp

outputs =

probas
indices

↳ Section 6:

Let's 1st head
Step ⑤ : The 1
mapping as
are mapped
Corresponding
emotion2i

emotion

In the last step, we convert the tensor of probs & tensor of associated indices from GPU-based to CPU-based, allowing for each tensor to be configured into a Numpy array. This then reverses both Numpy arrays such that the emotion with the highest prob. is first. This sets up the loop we'll run to display the images associated with the determined top K emotions experienced by the user.

```
outputs = torch.as_tensor(chain_model.predict_proba
                          (embeddings),
                          dtype=torch.float32)
probas, indices =
                torch.sort(outputs)

probas = probas.cpu().numpy()[0][::-1]
indices = indices.cpu().numpy()[0][::-1]
```

↳ Section 6: Display Results.

Let's 1st head back to the setup funct^n we discussed in step ⑤ : The 1st part of this funct^n initiates emotions2image-mapping as a dictionary & each of the 8 emotion labels are mapped with file paths. Corresponding to respective emotion GIFs. Then emotion2image-mapping iteratively filled with the emotion gifs themselves utilizing the file paths.

```
emotions2image_mapping = {
    'Anger' : './data/emotions/anger.gif',
    'Anticipation' : './data/emotions/anticipation.gif',
    'Disgust' : './data/emotions/disgust.gif',
    'Fear' : './data/emotions/fear.gif',
    'Joy' : './data/emotions/Joy.gif',
    'Sadness' : './data/emotions/sadness.gif',
    'Surprise' : './data/emotions/Surprise.gif',
    'Trust' : './data/emotions/trust.gif',
}
for key, value in emotions2image_mapping.items():
    with open(value, "rb") as f:
        emotions2image_mapping[key] = f.read()
```

↳ Now circling back to the score_sentence function, the call returns emotions2image_mapping to setup & is afterwards accessed in the loop inside score_sentence to display top k emotions to user as GIFs.

↳ The loop iterates through each of the top k emotions, with index & prob. associated with each emotion is extracted as tuple (index, p) at each iteration of the loop. Before the start of the loop, we use Streamlit to create k columns that will display top k emotion(s) to the user.

Text Generation

(1) Introduction to Text Generation

Here, you'll learn about Cohere's Command model & how an LLM chatbot works & get an introduction to Cohere's Chat endpoint.

↳ Chatbots brought LLMs into mainstream. LLMs have been around for a few years, but their adoption was largely limited to the AI community. The launch of AI-powered consumer chatbots has made LLMs accessible to the everyday user, & now they're a hot topic in tech & enterprise circles alike.

This text generation module teaches you how to build LLM chatbots using Cohere's Chat endpoint.

⇒ **Command Model**

Command is Cohere's flagship LLM. It generates a response given a prompt or message from a user. It is trained to follow user commands & to be instantly useful in practical business applications like summarization, copywriting, extraction & question answering.

Command has been trained with a large volume of multi-turn conversations to ensure that it excels at the various nuances associated with conversational language. It ranks at the top of the Holistic Evaluation of Language Models (HELM) benchmark, an evaluation leaderboard comparing LLMs on under no. of tasks.

Text Generation

(8) Introduction to Text Generation

Here, you'll learn about Cohere's Command model & how an LLM chatbot works & get an introduction to Cohere's Chat endpoint.

↳ Chatbots brought LLMs into mainstream. LLMs have been around for a few years, but their adoption was largely limited to the AI community. The launch of AI-powered consumer chatbots has made LLMs accessible to the everyday user, & now they're a hot topic in tech & enterprise circles alike.

This text generation module teaches you how to build LLM chatbots using Cohere's Chat endpoint.

⇒ **Command Model**

Command is Cohere's flagship LLM. It generates a response given a prompt or message from a user. It is trained to follow user commands & to be instantly useful in practical business applications like summarization, copywriting, extraction & question answering.

Command has been trained with a large volume of multi-turn conversations to ensure that it excels at the various nuances associated with conversational language. It ranks at the top of the Holistic Evaluation of Language Models (HELM) benchmark, an evaluation leaderboard comparing LLMs on under no. of tasks,

⇒ Command R & command R+ Models

Command R & Command R+ are designed to be the market leading family of models in the 'scalable' category that balance high efficiency with strong accuracy to enable enterprises to move from proof of concept into production-grade AI.

Here are some key features of command R:

- High-performance RAG: Retrieval-augmented generation (RAG) enables enterprises to give the model access to private knowledge that it otherwise would not have.
- Access to tools: Tools use enables enterprise developers to turn Command R into an engine for powering the automation of tasks and workflows that require using internal infrastructure like databases & software tools, as well as external tools like CRMs, search engines & more. Command R+ supports Multi-Step Tool Use which allows the model to combine multiple tools over multiple steps to accomplish difficult tasks.
- Low latency & high throughput: Command R targets the 'scalable' category of models that balance high performance with strong accuracy, enabling companies to move beyond proof of concept & into production

- 128 context length & lo

Command R features a
128K tokens in its i

- Strong capabilities

The model excels
business: English
Portuguese, Japa

- Model weights

Cohere for AI
version of
used for r

⇒ How an L

To understand ho
develop an understa
focuses on how to
by looking at ho
layers of co

↳ The Foundatio

The foundat
been fine-
a respons